utangtracker.ui/CustomerFrame.java • button "Logout" • lines 80-82

```
80          JButton out = Ui.dark("Logout");
81          out.setPreferredSize(new Dimension(100, 32));
82          out.addActionListener(e -> LoginFrame.logout(this));
```

utangtracker.ui/LoginFrame.java • button "Login" • lines 133-135

```
133         JButton login = Ui.primary("Login");
134         login.setPreferredSize(new Dimension(280, 42));
135         login.addActionListener(e -> doLogin());
```

utangtracker.ui/LoginFrame.java • button "Forgot password?" • lines 140-149

```
140         JLabel forgot = new JLabel("Forgot password?", SwingConstants.CENTER);
141         forgot.setFont(Ui.BODY);
142         forgot.setForeground(new Color(0x2E, 0x7D, 0x32));
143         forgot.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
144         forgot.addMouseListener(new MouseAdapter() {
145             @Override
146             public void mouseClicked(MouseEvent e) {
147                 resetPassword();
148             }
149         });
```

utangtracker.ui/LoginFrame.java • button "Don't have an account? Register" • lines 153-163

```
153         JLabel link = new JLabel("Don't have an account? Register", SwingConstants.CENTER);
154         link.setFont(Ui.BODY);
155         link.setForeground(new Color(0x2E, 0x7D, 0x32));
156         link.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
157         link.addMouseListener(new MouseAdapter() {
158             @Override
159             public void mouseClicked(MouseEvent e) {
160                 dispose();
161                 new RegisterFrame().setVisible(true);
162             }
163         });
```

utangtracker.ui/LoginFrame.java • resetPassword() • lines 169-173 • used by: Forgot password?

```
169     private void resetPassword() {
170         // MoneyTracker-style Reset PIN screen (see ResetPinFrame)
171         dispose();
172         new ResetPinFrame().setVisible(true);
173     }
```

utangtracker.ui/LoginFrame.java • doLogin() • lines 175-184 • used by: Login

```
175     private void doLogin() {
176         try {
177             User user = auth.login(username.getText(), new String(password.getPassword()));
178             Session.set(user);
179             dispose();
180             openHome(user);
181         } catch (AppException ex) {
182             JOptionPane.showMessageDialog(this, ex.getMessage(), "Login", JOptionPane.WARNING_MESSAGE);
183         }
184     }
```

utangtracker.ui/LoginFrame.java • openHome() • lines 186-196 • used by: Login

```
186     public static void openHome(User user) {
187         JFrame frame;
188         if (user.getRole() == Role.CUSTOMER) {
189             frame = new CustomerFrame();
190         } else if (user.getRole() == Role.STAFF) {
191             frame = new StaffFrame();
192         } else {
193             frame = new OwnerFrame();
194         }
195         frame.setVisible(true);
196     }
```

utangtracker.ui/LoginFrame.java • logout() • lines 198-202 • used by: CustomerFrame: Logout; OwnerFrame: Logout +2

```
198     public static void logout(JFrame current) {
199         Session.clear();
200         current.dispose();
201         new LoginFrame().setVisible(true);
202     }
```

utangtracker.ui/OwnerFrame.java • button "Logout" • lines 196-202

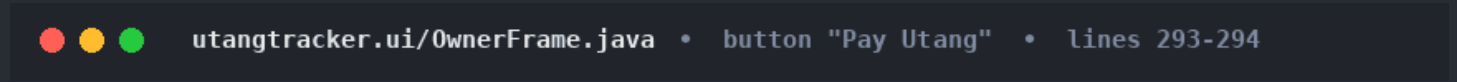
```
196         JLabel out = nav("Logout", "logout");
197         out.addMouseListener(new MouseAdapter() {
198             @Override
199             public void mouseClicked(MouseEvent e) {
200                 LoginFrame.logout(OwnerFrame.this);
201             }
202         });
```

utangtracker.ui/OwnerFrame.java • button "SansSerif" • lines 209-224

```
        final JLabel label = new JLabel(text);
        label.setFont(new Font("SansSerif", Font.BOLD, 14));
        label.setOpaque(true);
        label.setBackground(Color.WHITE);
        label.setForeground(Ui.TEXT);
        label.setBorder(javax.swing.BorderFactory.createEmptyBorder(12, 18, 12, 18));
        label.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
        label.setAlignmentX(LEFT_ALIGNMENT);
        label.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        if (!"logout".equals(page)) {
            label.addMouseListener(new MouseAdapter() {
                @Override
                public void mouseClicked(MouseEvent e) {
                    showPage(page);
                }
            });
```

utangtracker.ui/OwnerFrame.java • button "Add to Utang" • lines 291-292

```
291         JButton add = Ui.primary("Add to Utang");
292         add.addActionListener(e -> ownerAddUtang());
```


utangtracker.ui/OwnerFrame.java • button "Pay Utang" • lines 293-294
293 JButton pay = Ui.dark("Pay Utang");
294 pay.addActionListener(e -> recordPayment());

utangtracker.ui/OwnerFrame.java • button "Reset User PIN" • lines 295-296

```
295         JButton resetPin = Ui.ghost("Reset User PIN");
296         resetPin.addActionListener(e -> resetUserPin());
```

utangtracker.ui/OwnerFrame.java • button "Update Customer" • lines 347-348

```
347         JButton updateBtn = Ui.dark("Update Customer");
348         updateBtn.addActionListener(e -> updateSelectedCustomer(table));
```

utangtracker.ui/OwnerFrame.java • button "Create Staff" • lines 349-350

```
349         JButton createStaffBtn = Ui.primary("Create Staff");
350         createStaffBtn.addActionListener(e -> createStaff());
```

utangtracker.ui/OwnerFrame.java • button "Delete Customer" • lines 351-352

```
351         JButton deleteBtn = Ui.danger("Delete Customer");
352         deleteBtn.addActionListener(e -> deleteSelectedCustomer(table));
```

utangtracker.ui/OwnerFrame.java • button "table" • lines 357-386

```
357         table.addMouseListener(new MouseAdapter() {
358             @Override
359             public void mouseClicked(MouseEvent e) {
360                 int row = table.rowAtPoint(e.getPoint());
361                 int col = table.columnAtPoint(e.getPoint());
362                 if (row < 0) return;
363                 int modelRow = table.convertRowIndexToModel(row);
364                 String action = String.valueOf(accounts.getValueAt(modelRow, 6));
365                 if (col == 6 && "Approve".equals(action)) {
366                     String idStr = String.valueOf(accounts.getValueAt(modelRow, 0));
367                     int id = Integer.parseInt(idStr.replace("CUST-", ""));
368                     int confirm = JOptionPane.showConfirmDialog(OwnerFrame.this,
369                             "Approve this customer for utang?\nThey presented their valid ID?",
370                             "Approve Customer", JOptionPane.YES_NO_OPTION);
371                     if (confirm == JOptionPane.YES_OPTION) {
372                         try {
373                             auth.approveCustomer(id);
374                             JOptionPane.showMessageDialog(OwnerFrame.this, "Customer approved. They can now sign in.");
375                             reload();
376                         } catch (AppException ex) {
377                             JOptionPane.showMessageDialog(OwnerFrame.this, ex.getMessage(), "Approve", JOptionPane.WARNING_MESSAGE);
378                         }
379                     }
380                 } else if (col == 6 && "View".equals(action)) {
381                     String idStr = String.valueOf(accounts.getValueAt(modelRow, 0));
382                     int id = Integer.parseInt(idStr.replace("CUST-", ""));
383                     openAccountById(id);
384                 }
385             }
386         });
```

utangtracker.ui/OwnerFrame.java • button "table" • lines 566-590

```
566         JTable table = new JTable(pendingRows);
567         Ui.styleTable(table);
568         table.addMouseListener(new MouseAdapter() {
569             @Override
570             public void mouseClicked(MouseEvent e) {
571                 int row = table.rowAtPoint(e.getPoint());
572                 int col = table.columnAtPoint(e.getPoint());
573                 if (row >= 0 && col == 5) {
574                     String idStr = (String) pendingRows.getValueAt(row, 0);
575                     int id = Integer.parseInt(idStr.replace("CUST-", ""));
576                     int confirm = JOptionPane.showConfirmDialog(OwnerFrame.this,
577                             "Approve this customer for utang?\nThey presented their valid ID?",
578                             "Approve Customer", JOptionPane.YES_NO_OPTION);
579                     if (confirm == JOptionPane.YES_OPTION) {
580                         try {
581                             auth.approveCustomer(id);
582                             JOptionPane.showMessageDialog(OwnerFrame.this, "Customer approved. They can now sign in.");
583                             reload();
584                         } catch (AppException ex) {
585                             JOptionPane.showMessageDialog(OwnerFrame.this, ex.getMessage(), "Approve", JOptionPane.WARNING_MESSAGE);
586                         }
587                     }
588                 }
589             }
590         });
```

utangtracker.ui/OwnerFrame.java • button "Record Payment" • lines 722-723

```
722         JButton pay = Ui.primary("Record Payment");
723         pay.addActionListener(e -> recordPayment());
```

utangtracker.ui/OwnerFrame.java • showPage() • lines 229-236 • used by: SansSerif

```
229     private void showPage(String page) {
230         cards.show(content, page);
231         paintNav(dashboardNav, "dashboard".equals(page));
232         paintNav(accountsNav, "accounts".equals(page));
233         paintNav(logsNav, "logs".equals(page));
234         paintNav(paidNav, "paid".equals(page));
235         paintNav(pendingNav, "pending".equals(page));
236     }
```

utangtracker.ui/OwnerFrame.java • paintNav() • lines 238-241 • used by: SansSerif

```
238     private void paintNav(JLabel label, boolean active) {
239         label.setBackground(active ? new Color(0xE8, 0xF5, 0xE9) : Color.WHITE);
240         label.setForeground(active ? new Color(0x2E, 0x7D, 0x32) : Ui.TEXT);
241     }
```

utangtracker.ui/OwnerFrame.java • reload() • lines 626-712 • used by: Add to Utang; Delete Customer +4 • part 1/2

```
626     private void reload() {
627         try {
628             KpiSnapshot snap = utang.kpis();
629             totalUtang.setText(AppTime.peso(snap.getTotalUnpaid()));
630             activeCustomers.setText(String.valueOf(snap.getCustomersWithUtang()));
631             today.setText(String.valueOf(snap.getTransactionsToday()));
632             dashTotal.setText(AppTime.peso(snap.getTotalUnpaid()));
633             dashCustomers.setText(String.valueOf(snap.getCustomersWithUtang()));
634             dashToday.setText(String.valueOf(snap.getTransactionsToday()));
635             accountRows = utang.searchCustomers(accountSearch.getText());
636             filterOwnerCustomers();
637             accounts.setRowCount(0);
638 
639             // Active customers
640             for (CustomerBalance row : accountRows) {
641                 accounts.addRow(new Object[] {
642                         "CUST-" + row.getCustomer().getId(),
643                         row.getCustomer().getFirstName(),
644                         row.getCustomer().getLastName(),
645                         row.getCustomer().getContact(),
646                         AppTime.peso(row.getBalance()),
647                         "Active",
648                         "View"
649                 });
650             }
651 
652             // Pending customers (for approval)
653             List<User> pendingList = auth.pendingCustomers();
654             for (User p : pendingList) {
655                 String needle = accountSearch.getText().trim().toLowerCase();
656                 String hay = (p.getFirstName() + " " + p.getLastName() + " " + p.getUsername() + " " + p.getContact()).toLowerCase();
657                 if (needle.isEmpty() || hay.contains(needle)) {
658                     accounts.addRow(new Object[] {
659                             "CUST-" + p.getId(),
660                             p.getFirstName(),
661                             p.getLastName(),
662                             p.getContact(),
663                             "—",
664                             "Pending",
665                             "Approve"
666                     });
667                 }
668             }
669 
670             weekCollections.setText(AppTime.peso(utang.weekCollections()));
671             reportRows.setRowCount(0);
672             for (CustomerBalance row : utang.highestBalances()) {
673                 reportRows.addRow(new Object[] {
674                         "CUST-" + row.getCustomer().getId(),
675                         row.getCustomer().getFirstName(),
676                         row.getCustomer().getLastName(),
677                         AppTime.peso(row.getBalance())
678                 });
679             }
680             dailyRows.setRowCount(0);
```

utangtracker.ui/OwnerFrame.java • reload() • lines 626-712 • used by: Add to Utang; Delete Customer +4 • part 2/2

```
            allLedgerCache = utang.allHistory("", false);
            applyDateFilters();
            auditRows.setRowCount(0);
            allAuditCache = utang.auditLog();
            applyDateFilters();
            pendingRows.setRowCount(0);
            for (User pending : auth.pendingCustomers()) {
                pendingRows.addRow(new Object[] {
                        "CUST-" + pending.getId(),
                        pending.getFirstName(),
                        pending.getLastName(),
                        pending.getContact(),
                        pending.getUsername(),
                        "Approve"
                });
            }
            staffModel.clear();
            try {
                for (User s : utang.staff()) {
                    staffModel.addElement(s);
                }
            } catch (AppException ignored) {
                // Staff list may fail independently
            }
            dashLogs.setRowCount(0);
            paidLogs.setRowCount(0);
            logs.setRowCount(0);
            applyDateFilters();
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Owner", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/OwnerFrame.java • resetUserPin() • lines 729-750 • used by: Reset User PIN

```java
    private void resetUserPin() {
        String contact = JOptionPane.showInputDialog(this,
                "Enter the customer's contact number:",
                "Reset User PIN",
                JOptionPane.PLAIN_MESSAGE);
        if (contact == null || contact.trim().isEmpty()) {
            return;
        }
        try {
            User user = auth.resetPinByContact(contact.trim());
            JOptionPane.showMessageDialog(this,
                    "PIN reset successfully!\n\n"
                    + "Customer: " + user.getFirstName() + " " + user.getLastName() + "\n"
                    + "Contact: " + user.getContact() + "\n\n"
                    + "New PIN: 1234\n"
                    + "They can now log in using their contact number and PIN 1234.",
                    "Success",
                    JOptionPane.INFORMATION_MESSAGE);
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Reset PIN", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/OwnerFrame.java • createStaff() • lines 752-830 • used by: Create Staff • part 1/2

```java
    private void createStaff() {
        JTextField first = Ui.field("First name");
        JTextField last = Ui.field("Last name");
        JTextField contact = Ui.field("09XXXXXXXXX (optional)");
        JTextField user = Ui.field("Username");
        JPasswordField pass = Ui.password("Password (8-12 chars, capital, number, special)");

        first.setPreferredSize(new Dimension(320, 40));
        last.setPreferredSize(new Dimension(320, 40));
        contact.setPreferredSize(new Dimension(320, 40));
        user.setPreferredSize(new Dimension(320, 40));
        pass.setPreferredSize(new Dimension(320, 40));

        JPanel form = new JPanel(new GridBagLayout());
        form.setBorder(new javax.swing.border.EmptyBorder(8, 8, 8, 8));
        GridBagConstraints g = new GridBagConstraints();
        g.gridx = 0;
        g.gridy = 0;
        g.weightx = 1;
        g.fill = GridBagConstraints.HORIZONTAL;
        g.insets = new Insets(4, 0, 2, 0);
        g.anchor = GridBagConstraints.WEST;

        form.add(Ui.caption("First name"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(first, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Last name"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(last, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Contact Number (optional)"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(contact, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Username"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(user, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Password"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 4, 0);
        JPanel pwRow = new JPanel(new BorderLayout(6, 0));
```

utangtracker.ui/OwnerFrame.java • createStaff() • lines 752-830 • used by: Create Staff • part 2/2

```
807         pwRow.setOpaque(false);
808         pwRow.add(pass, BorderLayout.CENTER);
809         JCheckBox show = new JCheckBox("Show");
810         show.setOpaque(false);
811         show.setFont(Ui.BODY);
812         show.addActionListener(e -> pass.setEchoChar(show.isSelected() ? (char) 0 : '•'));
813         pwRow.add(show, BorderLayout.EAST);
814         form.add(pwRow, g);
815 
816         int ok = JOptionPane.showConfirmDialog(this, form, "Create Staff Account", JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
817         if (ok != JOptionPane.OK_OPTION) {
818             return;
819         }
820 
821         try {
822             User created = auth.createStaff(first.getText(), last.getText(), user.getText(),
823                     new String(pass.getPassword()), contact.getText());
824             JOptionPane.showMessageDialog(this,
825                     "Staff account created.\nUsername: " + created.getUsername(),
826                     "Success", JOptionPane.INFORMATION_MESSAGE);
827         } catch (AppException ex) {
828             JOptionPane.showMessageDialog(this, ex.getMessage(), "Create Staff", JOptionPane.WARNING_MESSAGE);
829         }
830     }
```

utangtracker.ui/OwnerFrame.java • deleteSelectedCustomer() • lines 832-868 • used by: Delete Customer

```java
    private void deleteSelectedCustomer(JTable table) {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a customer first.");
            return;
        }
        int modelRow = table.convertRowIndexToModel(row);
        String idStr = String.valueOf(accounts.getValueAt(modelRow, 0));
        int id = Integer.parseInt(idStr.replace("CUST-", ""));
        String name = accounts.getValueAt(modelRow, 1) + " " + accounts.getValueAt(modelRow, 2);

        // Step 1: yes / no
        int first = JOptionPane.showConfirmDialog(this,
                "Delete customer " + name + " (" + idStr + ")?\n\n"
                + "Their account and ALL of their transaction history will be permanently removed.\n"
                + "This cannot be undone.",
                "Delete Customer", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
        if (first != JOptionPane.YES_OPTION) {
            return;
        }
        // Step 2: type DELETE to confirm
        String typed = JOptionPane.showInputDialog(this,
                "To confirm, type DELETE below:", "Confirm Delete", JOptionPane.WARNING_MESSAGE);
        if (typed == null || !"DELETE".equals(typed.trim())) {
            if (typed != null) {
                JOptionPane.showMessageDialog(this, "Not deleted. You must type DELETE exactly.");
            }
            return;
        }
        try {
            auth.deleteCustomer(id);
            JOptionPane.showMessageDialog(this, name + " (" + idStr + ") was deleted.");
            reload();
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Delete Customer", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/OwnerFrame.java • updateSelectedCustomer() • lines 870-938 • used by: Update Customer • part 1/2

```java
    private void updateSelectedCustomer(JTable table) {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a customer first.");
            return;
        }
        int modelRow = table.convertRowIndexToModel(row);
        String status = String.valueOf(accounts.getValueAt(modelRow, 5));
        if ("Pending".equals(status)) {
            JOptionPane.showMessageDialog(this, "Approve the customer first before updating details.");
            return;
        }
        String idStr = String.valueOf(accounts.getValueAt(modelRow, 0));
        int id = Integer.parseInt(idStr.replace("CUST-", ""));
        String currentFirst = String.valueOf(accounts.getValueAt(modelRow, 1));
        String currentLast = String.valueOf(accounts.getValueAt(modelRow, 2));
        String currentContact = String.valueOf(accounts.getValueAt(modelRow, 3));

        JTextField first = Ui.field("");
        JTextField last = Ui.field("");
        JTextField contact = Ui.field("");
        first.setText(currentFirst);
        last.setText(currentLast);
        contact.setText(currentContact);
        first.setPreferredSize(new Dimension(320, 40));
        last.setPreferredSize(new Dimension(320, 40));
        contact.setPreferredSize(new Dimension(320, 40));

        JPanel form = new JPanel(new GridBagLayout());
        form.setBorder(new javax.swing.border.EmptyBorder(8, 8, 8, 8));
        GridBagConstraints g = new GridBagConstraints();
        g.gridx = 0;
        g.gridy = 0;
        g.weightx = 1;
        g.fill = GridBagConstraints.HORIZONTAL;
        g.insets = new Insets(4, 0, 2, 0);
        g.anchor = GridBagConstraints.WEST;

        form.add(Ui.caption("First name"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(first, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Last name"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 8, 0);
        form.add(last, g);

        g.gridy++;
        g.insets = new Insets(4, 0, 2, 0);
        form.add(Ui.caption("Contact Number"), g);
        g.gridy++;
        g.insets = new Insets(0, 0, 4, 0);
```

utangtracker.ui/OwnerFrame.java • updateSelectedCustomer() • lines 870-938 • used by: Update Customer • part 2/2

```
925         form.add(contact, g);
926
927         int ok = JOptionPane.showConfirmDialog(this, form, "Update Customer " + idStr, JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
928         if (ok != JOptionPane.OK_OPTION) {
929             return;
930         }
931         try {
932             auth.updateCustomerInfo(id, first.getText(), last.getText(), contact.getText());
933             JOptionPane.showMessageDialog(this, "Customer information updated.");
934             reload();
935         } catch (AppException ex) {
936             JOptionPane.showMessageDialog(this, ex.getMessage(), "Update", JOptionPane.WARNING_MESSAGE);
937         }
938     }
```

utangtracker.ui/OwnerFrame.java • ownerAddUtang() • lines 954-988 • used by: Add to Utang

```java
    private void ownerAddUtang() {
        try {
            CustomerBalance selected = ownerList.getSelectedValue();
            if (selected == null) {
                JOptionPane.showMessageDialog(this, "Select a customer first.");
                return;
            }
            double q = Double.parseDouble(ownerQty.getText().trim());
            double u = Double.parseDouble(ownerUnit.getText().trim());
            LedgerEntry entry = utang.addUtang(selected.getCustomer().getId(), ownerItem.getText(),
                    q, u, "");
            ownerItem.setText("");
            ownerQty.setText("1");
            ownerUnit.setText("");
            ownerAmount.setText("0.00");
            reload();
            User cust = selected.getCustomer();
            String receipt = "UTANG TRACKER SYSTEM - RECEIPT\n"
                    + "----------------------------\n"
                    + "Date: " + entry.getCreatedAt() + "\n"
                    + "Customer: " + cust.getFirstName() + " " + cust.getLastName() + " (CUST-" + cust.getId() + ")\n"
                    + "Item: " + entry.getItemDescription() + "\n"
                    + "Quantity: " + String.format("%.2f", entry.getQuantity()) + "\n"
                    + "Unit Price: " + AppTime.peso(entry.getUnitPrice()) + "\n"
                    + "Total Amount: " + AppTime.peso(entry.getAmount()) + "\n"
                    + "Status: Unpaid (Utang)\n"
                    + "----------------------------\n"
                    + "Thank you!";
            JOptionPane.showMessageDialog(this, receipt, "Receipt", JOptionPane.INFORMATION_MESSAGE);
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Add to Utang", JOptionPane.WARNING_MESSAGE);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Enter valid quantity and unit price.", "Add to Utang", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/OwnerFrame.java • recordPayment() • lines 990-1016 • used by: Pay Utang; Record Payment

```
990     private void recordPayment() {
991         CustomerBalance selected = ownerList.getSelectedValue();
992         if (selected == null) {
993             JOptionPane.showMessageDialog(this, "Select a customer first.");
994             return;
995         }
996         String typed = payAmount.getText().trim();
997         if (typed.isEmpty()) {
998             typed = JOptionPane.showInputDialog(this, "Payment amount for "
999                     + selected.getCustomer().getFirstName() + " "
1000                    + selected.getCustomer().getLastName());
1001        }
1002        if (typed == null || typed.trim().isEmpty()) {
1003            return;
1004        }
1005        try {
1006            utang.acceptPayment(selected.getCustomer().getId(),
1007                    Double.parseDouble(typed.trim()), "Owner payment");
1008            payAmount.setText("");
1009            reload();
1010            JOptionPane.showMessageDialog(this, "Payment recorded.");
1011        } catch (AppException ex) {
1012            JOptionPane.showMessageDialog(this, ex.getMessage(), "Pay Utang", JOptionPane.WARNING_MESSAGE);
1013        } catch (NumberFormatException ex) {
1014            JOptionPane.showMessageDialog(this, "Enter a valid amount.", "Pay Utang", JOptionPane.WARNING_MESSAGE);
1015        }
1016    }
```

utangtracker.ui/OwnerFrame.java • openAccountById() • lines 1018-1058 • used by: table

```java
    private void openAccountById(int customerId) {
        CustomerBalance found = null;
        if (accountRows != null) {
            for (CustomerBalance row : accountRows) {
                if (row.getCustomer().getId() == customerId) {
                    found = row;
                    break;
                }
            }
        }
        if (found == null) {
            JOptionPane.showMessageDialog(this, "Customer not found.");
            return;
        }
        DefaultTableModel history = new DefaultTableModel(new Object[] {"Created Date&Time", "Item Description", "Amount", "Note", "Status"}, 0);
        try {
            List<LedgerEntry> rows = utang.historyFor(found.getCustomer().getId());
            for (LedgerEntry entry : rows) {
                history.addRow(new Object[] {
                        entry.getCreatedAt(),
                        entry.getItemDescription(),
                        AppTime.peso(entry.getAmount()),
                        entry.getNote() == null || entry.getNote().isEmpty() ? "—" : entry.getNote(),
                        entry.isUtang() ? "Unpaid" : "Paid"
                });
            }
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "View", JOptionPane.WARNING_MESSAGE);
            return;
        }
        JTable table = new JTable(history);
        Ui.styleTable(table);
        Ui.enableLongCellPopup(table);
        table.setPreferredScrollableViewportSize(new Dimension(520, 160));
        JPanel form = new JPanel(new BorderLayout(0, 8));
        form.add(new JLabel("CUST-" + found.getCustomer().getId() + "  "
                + found.getCustomer().getFirstName() + " " + found.getCustomer().getLastName()
                + "    " + AppTime.peso(found.getBalance())), BorderLayout.NORTH);
        form.add(new JScrollPane(table), BorderLayout.CENTER);
        JOptionPane.showMessageDialog(this, form, "CUST-" + found.getCustomer().getId(), JOptionPane.INFORMATION_MESSAGE);
    }
```

utangtracker.ui/OwnerFrame.java • applyDateFilters() • lines 1094-1178 • used by: Add to Utang; Delete Customer +4 • part 1/2

```java
    private void applyDateFilters() {
        String logsNeedle = logsDate.getText().trim();
        String paidNeedle = paidDate.getText().trim();
        String reportsNeedle = reportsDate.getText().trim();
        String auditNeedle = auditDate.getText().trim();

        logs.setRowCount(0);
        paidLogs.setRowCount(0);
        dailyRows.setRowCount(0);
        auditRows.setRowCount(0);
        dashLogs.setRowCount(0);

        double todayUtang = 0;
        double todayPay = 0;
        String today = AppTime.todayPrefix();

        for (LedgerEntry row : allLedgerCache) {
            String when = row.getCreatedAt();
            boolean matchLogs = logsNeedle.isEmpty() || when.startsWith(logsNeedle);
            boolean matchPaid = paidNeedle.isEmpty() || when.startsWith(paidNeedle);
            boolean matchReports = reportsNeedle.isEmpty() || when.startsWith(reportsNeedle);

            if (matchLogs) {
                logs.addRow(new Object[] {
                        when,
                        "CUST-" + row.getCustomerId(),
                        row.getCustomerFirstName(),
                        row.getCustomerLastName(),
                        (row.getStaffFirstName() == null ? "System" : row.getStaffFirstName() + " " + row.getStaffLastName()),
                        row.getItemDescription(),
                        AppTime.peso(row.getAmount()),
                        row.isUtang() ? "Added utang" : "Payment",
                        row.getNote() == null || row.getNote().isEmpty() ? "—" : row.getNote()
                });
            }
            if (!row.isUtang() && matchPaid) {
                paidLogs.addRow(new Object[] {
                        when,
                        "CUST-" + row.getCustomerId(),
                        row.getCustomerFirstName(),
                        row.getCustomerLastName(),
                        row.getStaffFirstName() == null ? "System" : row.getStaffFirstName() + " " + row.getStaffLastName(),
                        AppTime.peso(row.getAmount()),
                        row.getNote() == null || row.getNote().isEmpty() ? "—" : row.getNote()
                });
            }
            if (matchReports) {
                dailyRows.addRow(new Object[] {
                        when,
                        row.getCustomerFirstName() + " " + row.getCustomerLastName(),
                        row.getItemDescription(),
                        String.format("%.2f", row.getQuantity()),
                        AppTime.peso(row.getUnitPrice()),
                        AppTime.peso(row.getAmount()),
                        (row.getStaffFirstName() == null ? "System" : row.getStaffFirstName() + " " + row.getStaffLastName()),
```

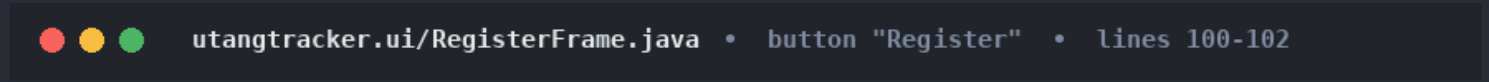
utangtracker.ui/OwnerFrame.java • applyDateFilters() • lines 1094-1178 • used by: Add to Utang; Delete Customer +4 • part 2/2

```
                        row.getType()
                });
            }
            // Dashboard recent still shows today's activity
            if (when.startsWith(today)) {
                dashLogs.addRow(new Object[] {
                        when,
                        row.getCustomerFirstName() + " " + row.getCustomerLastName(),
                        row.getItemDescription(),
                        AppTime.peso(row.getAmount()),
                        (row.getStaffFirstName() == null ? "System" : row.getStaffFirstName() + " " + row.getStaffLastName()),
                        row.isUtang() ? "Added utang" : "Payment"
                });
                if (row.isUtang()) {
                    todayUtang += row.getAmount();
                } else {
                    todayPay += row.getAmount();
                }
            }
        }
        dailyUtangTotal.setText(AppTime.peso(todayUtang));
        dailyPaymentTotal.setText(AppTime.peso(todayPay));

        for (String[] row : allAuditCache) {
            String when = row.length > 0 ? row[0] : "";
            if (auditNeedle.isEmpty() || when.startsWith(auditNeedle)) {
                auditRows.addRow(row);
            }
        }
    }
```

utangtracker.ui/OwnerFrame.java • filterOwnerCustomers() • lines 1180-1202 • used by: Add to Utang; Delete Customer +4

```
1180    private void filterOwnerCustomers() {
1181        if (accountRows == null) {
1182            return;
1183        }
1184        String needle = ownerCustomerSearch.getText().trim().toLowerCase();
1185        CustomerBalance previous = ownerList.getSelectedValue();
1186        ownerCustomers.clear();
1187        for (CustomerBalance row : accountRows) {
1188            String blob = (row.getCustomer().getFirstName() + " " + row.getCustomer().getLastName()
1189                    + " " + row.getCustomer().getId() + " " + row.getCustomer().getContact()).toLowerCase();
1190            if (needle.isEmpty() || blob.contains(needle)) {
1191                ownerCustomers.addElement(row);
1192            }
1193        }
1194        if (previous != null) {
1195            for (int i = 0; i < ownerCustomers.size(); i++) {
1196                if (ownerCustomers.get(i).getCustomer().getId() == previous.getCustomer().getId()) {
1197                    ownerList.setSelectedIndex(i);
1198                    break;
1199                }
1200            }
1201        }
1202    }
```


utangtracker.ui/RegisterFrame.java • button "Register" • lines 100-102
100 JButton register = Ui.primary("Register");
101 register.setPreferredSize(new Dimension(280, 42));
102 register.addActionListener(e -> save());

utangtracker.ui/RegisterFrame.java • button "Back to Login" • lines 107-117

```
107         JLabel back = new JLabel("Back to Login", SwingConstants.CENTER);
108         back.setFont(Ui.BODY);
109         back.setForeground(new Color(0x2E, 0x7D, 0x32));
110         back.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
111         back.addMouseListener(new MouseAdapter() {
112             @Override
113             public void mouseClicked(MouseEvent e) {
114                 dispose();
115                 new LoginFrame().setVisible(true);
116             }
117         });
```

utangtracker.ui/RegisterFrame.java • save() • lines 168-183 • used by: Register

```
168     private void save() {
169         try {
170             utangtracker.model.User created = auth.registerCustomer(firstName.getText(), lastName.getText(), username.getText(),
171                     new String(password.getPassword()), new String(password.getPassword()), contact.getText(), new String(pin.getPassword()));
172             JOptionPane.showMessageDialog(this,
173                     "Account created successfully!\n\nYour ID is CUST-" + created.getId() + ".\n\n" +
174                     "Please present your valid ID to the store owner/admin for utang approval.\n" +
175                     "You will be able to sign in after approval.",
176                     "Registration Complete — Valid ID Required",
177                     JOptionPane.INFORMATION_MESSAGE);
178             dispose();
179             new LoginFrame().setVisible(true);
180         } catch (AppException ex) {
181             JOptionPane.showMessageDialog(this, ex.getMessage(), "Register", JOptionPane.WARNING_MESSAGE);
182         }
183     }
```

utangtracker.ui/ResetPinFrame.java • button "back" • lines 56-56

```
56          form.back.addActionListener(e -> backToLogin());
```

utangtracker.ui/ResetPinFrame.java • button "reset" • lines 58-58

```
58          form.reset.addActionListener(e -> doReset());
```

utangtracker.ui/ResetPinFrame.java • doReset() • lines 72-85 • used by: reset

```
72      private void doReset() {
73          try {
74              auth.resetPassword(form.username.getText(),
75                      new String(form.pin.getPassword()),
76                      new String(form.newPassword.getPassword()),
77                      new String(form.confirm.getPassword()));
78              JOptionPane.showMessageDialog(this,
79                      "Password updated. Sign in with your new password.",
80                      "Reset PIN", JOptionPane.INFORMATION_MESSAGE);
81              backToLogin();
82          } catch (AppException ex) {
83              JOptionPane.showMessageDialog(this, ex.getMessage(), "Reset PIN", JOptionPane.WARNING_MESSAGE);
84          }
85      }
```

utangtracker.ui/ResetPinFrame.java • backToLogin() • lines 87-90 • used by: back; reset

```
87      private void backToLogin() {
88          dispose();
89          new LoginFrame().setVisible(true);
90      }
```

utangtracker.ui/StaffFrame.java • button "Logout" • lines 95-97

```
95          JButton out = Ui.dark("Logout");
96          out.setPreferredSize(new Dimension(100, 32));
97          out.addActionListener(e -> LoginFrame.logout(this));
```

utangtracker.ui/StaffFrame.java • button "Create Customer" • lines 98-100

```
98          JButton create = Ui.dark("Create Customer");
99          create.setPreferredSize(new Dimension(140, 32));
100         create.addActionListener(e -> createCustomer());
```

utangtracker.ui/StaffFrame.java • button "Add to Utang" • lines 204-206

```
204         JButton add = Ui.primary("Add to Utang");
205         add.setPreferredSize(new Dimension(140, 40));
206         add.addActionListener(e -> addUtang());
```

utangtracker.ui/StaffFrame.java • button "Pay Utang" • lines 207-209

```
207         JButton pay = Ui.dark("Pay Utang");
208         pay.setPreferredSize(new Dimension(130, 40));
209         pay.addActionListener(e -> payUtang());
```

utangtracker.ui/StaffFrame.java • reloadCustomers() • lines 229-236 • used by: Add to Utang; Create Customer +1

```java
    private void reloadCustomers() {
        try {
            loaded = utang.searchCustomers("");
            filter();
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Staff POS", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/StaffFrame.java • filter() • lines 238-257 • used by: Add to Utang; Create Customer +1

```
238     private void filter() {
239         String needle = search.getText().trim().toLowerCase();
240         CustomerBalance previous = list.getSelectedValue();
241         customers.clear();
242         for (CustomerBalance row : loaded) {
243             String blob = (row.getCustomer().getFirstName() + " " + row.getCustomer().getLastName()
244                     + " " + row.getCustomer().getId() + " " + row.getCustomer().getContact()).toLowerCase();
245             if (needle.isEmpty() || blob.contains(needle)) {
246                 customers.addElement(row);
247             }
248         }
249         if (previous != null) {
250             for (int i = 0; i < customers.size(); i++) {
251                 if (customers.get(i).getCustomer().getId() == previous.getCustomer().getId()) {
252                     list.setSelectedIndex(i);
253                     return;
254                 }
255             }
256         }
257     }
```

utangtracker.ui/StaffFrame.java • createCustomer() • lines 301-348 • used by: Create Customer

```java
    private void createCustomer() {
        JTextField first = Ui.field("First name");
        JTextField last = Ui.field("Last name");
        JTextField contact = Ui.field("09XXXXXXXXX");
        JTextField user = Ui.field("Username");
        JPasswordField pass = Ui.password("Password (8-12, capital, number, special)");
        JPasswordField pin = Ui.password("4-digit Recovery PIN");

        JPanel form = new JPanel(new java.awt.GridLayout(0, 1, 0, 6));
        form.add(new JLabel("First name"));
        form.add(first);
        form.add(new JLabel("Last name"));
        form.add(last);
        form.add(new JLabel("Contact Number"));
        form.add(contact);
        form.add(new JLabel("Username"));
        form.add(user);
        form.add(new JLabel("Password"));
        form.add(pass);
        form.add(new JLabel("4-digit Recovery PIN"));
        form.add(pin);

        int ok = JOptionPane.showConfirmDialog(this, form, "Create Customer Account", JOptionPane.OK_CANCEL_OPTION);
        if (ok != JOptionPane.OK_OPTION) {
            return;
        }

        int idConfirm = JOptionPane.showConfirmDialog(this,
                "Has the customer presented a valid ID for utang approval?",
                "Valid ID Confirmation", JOptionPane.YES_NO_OPTION);
        if (idConfirm != JOptionPane.YES_OPTION) {
            JOptionPane.showMessageDialog(this, "Account creation cancelled. Valid ID is required.");
            return;
        }

        try {
            utangtracker.model.User created = auth.registerCustomer(
                    first.getText(), last.getText(), user.getText(),
                    new String(pass.getPassword()), new String(pass.getPassword()),
                    contact.getText(), new String(pin.getPassword()));
            JOptionPane.showMessageDialog(this,
                    "Customer account created.\nID: CUST-" + created.getId() + "\nUsername: " + created.getUsername(),
                    "Success", JOptionPane.INFORMATION_MESSAGE);
            reloadCustomers();
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Create Customer", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/StaffFrame.java • addUtang() • lines 350-371 • used by: Add to Utang

```java
    private void addUtang() {
        CustomerBalance row = list.getSelectedValue();
        if (row == null) {
            JOptionPane.showMessageDialog(this, "Select a customer first.");
            return;
        }
        try {
            double q = Double.parseDouble(quantity.getText().trim());
            double u = Double.parseDouble(unitPrice.getText().trim());
            LedgerEntry entry = utang.addUtang(row.getCustomer().getId(), item.getText(), q, u, "");
            item.setText("");
            quantity.setText("1");
            unitPrice.setText("");
            amount.setText("0.00");
            reloadCustomers();
            showReceipt(entry, row.getCustomer());
        } catch (AppException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Add to Utang", JOptionPane.WARNING_MESSAGE);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Enter valid quantity and unit price.", "Add to Utang", JOptionPane.WARNING_MESSAGE);
        }
    }
```

utangtracker.ui/StaffFrame.java • showReceipt() • lines 373-386 • used by: Add to Utang

```
373     private void showReceipt(LedgerEntry entry, User customer) {
374         String receipt = "UTANG TRACKER SYSTEM - RECEIPT\n"
375                 + "----------------------------\n"
376                 + "Date: " + entry.getCreatedAt() + "\n"
377                 + "Customer: " + customer.getFirstName() + " " + customer.getLastName() + " (CUST-" + customer.getId() + ")\n"
378                 + "Item: " + entry.getItemDescription() + "\n"
379                 + "Quantity: " + String.format("%.2f", entry.getQuantity()) + "\n"
380                 + "Unit Price: " + AppTime.peso(entry.getUnitPrice()) + "\n"
381                 + "Total Amount: " + AppTime.peso(entry.getAmount()) + "\n"
382                 + "Status: Unpaid (Utang)\n"
383                 + "----------------------------\n"
384                 + "Thank you!";
385         JOptionPane.showMessageDialog(this, receipt, "Receipt", JOptionPane.INFORMATION_MESSAGE);
386     }
```

utangtracker.ui/StaffFrame.java • payUtang() • lines 388-409 • used by: Pay Utang

```
388     private void payUtang() {
389         CustomerBalance row = list.getSelectedValue();
390         if (row == null) {
391             JOptionPane.showMessageDialog(this, "Select a customer first.");
392             return;
393         }
394         String typed = JOptionPane.showInputDialog(this, "Payment amount for "
395                 + row.getCustomer().getFirstName() + " " + row.getCustomer().getLastName()
396                 + " (" + AppTime.peso(row.getBalance()) + ")");
397         if (typed == null || typed.trim().isEmpty()) {
398             return;
399         }
400         try {
401             utang.acceptPayment(row.getCustomer().getId(), Double.parseDouble(typed.trim()), "Staff payment");
402             reloadCustomers();
403             JOptionPane.showMessageDialog(this, "Payment recorded.");
404         } catch (AppException ex) {
405             JOptionPane.showMessageDialog(this, ex.getMessage(), "Pay Utang", JOptionPane.WARNING_MESSAGE);
406         } catch (NumberFormatException ex) {
407             JOptionPane.showMessageDialog(this, "Enter a valid amount.", "Pay Utang", JOptionPane.WARNING_MESSAGE);
408         }
409     }
```

utangtracker.ui/Ui.java • button "Logout" • lines 308-315

```
308         JButton out = dark("Logout");
309         out.setIcon(circleIcon("out", 16, Color.WHITE, GREEN_DARK));
310         out.setPreferredSize(new java.awt.Dimension(118, 34));
311         out.addActionListener(new java.awt.event.ActionListener() {
312             public void actionPerformed(java.awt.event.ActionEvent e) {
313                 LoginFrame.logout(frame);
314             }
315         });
```

utangtracker.service/AuthService.java • login() • lines 15-46 • used by: LoginFrame: Login

```java
    public User login(String username, String password) throws AppException {
        if (isBlank(username) || isBlank(password)) {
            throw new AppException("Enter both username and password.");
        }
        try {
            // Allow login with contact number + recovery PIN (after owner resets PIN)
            String digits = username.replaceAll("[^0-9]", "");
            if (digits.length() >= 10 && password.trim().matches("\\d{4}")) {
                User byContact = database.findCustomerByContact(digits);
                if (byContact != null && database.matchesRecoveryPin(byContact.getId(), password.trim())) {
                    if (!byContact.isActive()) {
                        throw new AppException("Your account is waiting for the owner's approval. Please present a valid ID to the owner.");
                    }
                    return byContact;
                }
            }

            String[] credentials = database.credentials(username);
            if (credentials == null || !PasswordUtil.matches(password, credentials[1], credentials[0])) {
                throw new AppException("Wrong username or password.");
            }
            if (!"1".equals(credentials[2])) {
                if (Role.CUSTOMER.name().equals(credentials[3])) {
                    throw new AppException("Your account is waiting for the owner's approval. Please present a valid ID to the owner.");
                }
                throw new AppException("This account is inactive. Ask the owner to restore it.");
            }
            return database.findByUsername(username);
        } catch (SQLException ex) {
            throw new AppException("Cannot sign in right now. Please try again.");
        }
    }
```

utangtracker.service/AuthService.java • resetPinByContact() • lines 48-65 • used by: OwnerFrame: Reset User PIN

```java
    /** Owner resets a customer's recovery PIN to 1234 by contact number. */
    public User resetPinByContact(String contact) throws AppException {
        requireOwner();
        if (contact == null || contact.replaceAll("[^0-9]", "").length() < 10) {
            throw new AppException("Enter a valid contact number (at least 10 digits).");
        }
        try {
            User user = database.resetPinByContact(contact, "1234");
            if (user == null) {
                throw new AppException("No active customer found with that contact number.");
            }
            database.audit(Session.get().getId(), actor(), "UPDATE", "CUST-" + user.getId(),
                    "Reset recovery PIN for " + user.getFirstName() + " " + user.getLastName());
            return user;
        } catch (SQLException ex) {
            throw new AppException("Cannot reset the PIN right now.");
        }
    }
```

utangtracker.service/AuthService.java • resetPassword() • lines 67-91 • used by: ResetPinFrame: reset

```java
    public void resetPassword(String username, String pin, String newPassword, String confirm) throws AppException {
        if (isBlank(username)) {
            throw new AppException("Enter the username.");
        }
        if (pin == null || !pin.trim().matches("\\d{4}")) {
            throw new AppException("Recovery PIN must be exactly 4 numbers.");
        }
        if (isBlank(newPassword) || !isStrongPassword(newPassword)) {
            throw new AppException("New password must be 8–12 characters and include at least one capital letter, one number, and one special character.");
        }
        if (!newPassword.equals(confirm)) {
            throw new AppException("Password and confirmation do not match.");
        }
        try {
            int result = database.resetPassword(username, pin.trim(), newPassword);
            if (result == Database.RESET_LOCKED) {
                throw new AppException("Too many wrong PIN tries. Try again in 15 minutes.");
            }
            if (result != Database.RESET_OK) {
                throw new AppException("Username or recovery PIN is not correct, or this account cannot use PIN recovery. Staff and owner accounts: ask the owner.");
            }
        } catch (SQLException ex) {
            throw new AppException("Cannot reset the password right now.");
        }
    }
```

utangtracker.service/AuthService.java • registerCustomer() • lines 93-109 • used by: RegisterFrame: Register; StaffFrame: Create Customer

```java
    public User registerCustomer(String firstName, String lastName, String username, String password, String confirm, String contact, String pin) throws AppException {
        validatePerson(firstName, lastName, username, password, confirm, contact, true);
        if (pin == null || !pin.trim().matches("\\d{4}")) {
            throw new AppException("Recovery PIN must be exactly 4 numbers.");
        }
        try {
            if (database.usernameExists(username)) {
                throw new AppException("That username is already taken.");
            }
            int id = database.insertUser(firstName, lastName, username, password, contact, Role.CUSTOMER);
            database.saveRecoveryPin(id, pin.trim());
            database.audit(id, firstName.trim() + " " + lastName.trim(), "CREATE", "CUST-" + id, "Registered customer account");
            return database.findUser(id);
        } catch (SQLException ex) {
            throw new AppException("Cannot create the account right now.");
        }
    }
```

utangtracker.service/AuthService.java • createStaff() • lines 111-124 • used by: OwnerFrame: Create Staff

```java
    public User createStaff(String firstName, String lastName, String username, String password, String contact) throws AppException {
        requireOwner();
        validatePerson(firstName, lastName, username, password, password, contact, false);
        try {
            if (database.usernameExists(username)) {
                throw new AppException("That username is already taken.");
            }
            int id = database.insertUser(firstName, lastName, username, password, contact, Role.STAFF);
            database.audit(Session.get().getId(), actor(), "CREATE", "STAFF-" + id, "Created staff account " + username.trim().toLowerCase());
            return database.findUser(id);
        } catch (SQLException ex) {
            throw new AppException("Cannot create the staff account right now.");
        }
    }
```

utangtracker.service/AuthService.java • updateCustomerInfo() • lines 126-149 • used by: OwnerFrame: Update Customer

```java
    public void updateCustomerInfo(int userId, String firstName, String lastName, String contact) throws AppException {
        requireOwner();
        if (firstName == null || firstName.trim().length() < 2) {
            throw new AppException("Enter a valid first name.");
        }
        if (lastName == null || lastName.trim().length() < 2) {
            throw new AppException("Enter a valid last name.");
        }
        String digits = contact == null ? "" : contact.replaceAll("[^0-9]", "");
        if (digits.length() < 10 || digits.length() > 13) {
            throw new AppException("Enter a valid contact number (10–13 digits).");
        }
        try {
            User user = database.findUser(userId);
            if (user == null || user.getRole() != Role.CUSTOMER) {
                throw new AppException("Only customer accounts can be updated here.");
            }
            database.updateCustomerInfo(userId, firstName, lastName, contact);
            database.audit(Session.get().getId(), Session.get().getFirstName() + " " + Session.get().getLastName(),
                    "UPDATE", "CUST-" + userId, "Updated customer info");
        } catch (SQLException ex) {
            throw new AppException("Cannot update customer information.");
        }
    }
```

utangtracker.service/AuthService.java • approveCustomer() • lines 170-183 • used by: OwnerFrame: table

```java
    public void approveCustomer(int userId) throws AppException {
        requireOwner();
        try {
            User user = database.findUser(userId);
            if (user == null || user.getRole() != Role.CUSTOMER) {
                throw new AppException("Only customer accounts can be approved here.");
            }
            database.setActive(userId, true);
            database.audit(Session.get().getId(), Session.get().getFirstName() + " " + Session.get().getLastName(),
                    "APPROVE", "CUST-" + userId, "Approved customer registration");
        } catch (SQLException ex) {
            throw new AppException("Cannot approve that customer.");
        }
    }
```

utangtracker.service/AuthService.java • deleteCustomer() • lines 185-204 • used by: OwnerFrame: Delete Customer

```java
    /** Owner deletes a customer account. Refused while the customer still owes money. */
    public void deleteCustomer(int userId) throws AppException {
        requireOwner();
        try {
            User user = database.findUser(userId);
            if (user == null || user.getRole() != Role.CUSTOMER) {
                throw new AppException("Only customer accounts can be deleted here.");
            }
            double balance = database.balanceOf(userId);
            if (balance > 0.009) {
                throw new AppException("This customer still owes " + String.format("\u20B1%.2f", balance)
                        + ".\nRecord their payment first, then delete the account.");
            }
            database.deleteCustomer(userId);
            database.audit(Session.get().getId(), actor(), "DELETE", "CUST-" + userId,
                    "Deleted customer account " + user.getFirstName() + " " + user.getLastName());
        } catch (SQLException ex) {
            throw new AppException("Cannot delete that customer right now.");
        }
    }
```

utangtracker.service/AuthService.java • pendingCustomers() • lines 206-213 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```
206     public List<User> pendingCustomers() throws AppException {
207         requireOwner();
208         try {
209             return database.listPendingCustomers();
210         } catch (SQLException ex) {
211             throw new AppException("Cannot load pending customers.");
212         }
213     }
```

utangtracker.service/AuthService.java • validatePerson() • lines 230-253 • used by: OwnerFrame: Create Staff; RegisterFrame: Register +1

```java
    private void validatePerson(String firstName, String lastName, String username, String password, String confirm, String contact, boolean customer) throws AppException {
        if (isBlank(firstName) || firstName.trim().length() < 2) {
            throw new AppException("Enter a first name.");
        }
        if (isBlank(lastName) || lastName.trim().length() < 2) {
            throw new AppException("Enter a last name.");
        }
        if (isBlank(username) || !username.trim().matches("[A-Za-z0-9._]{4,20}")) {
            throw new AppException("Username must be 4–20 letters, numbers, dots, or underscores.");
        }
        if (isBlank(password) || !isStrongPassword(password)) {
            throw new AppException("Password must be 8–12 characters and include at least one capital letter, one number, and one special character.");
        }
        if (!password.equals(confirm)) {
            throw new AppException("Password and confirmation do not match.");
        }
        String digits = contact == null ? "" : contact.replaceAll("[^0-9]", "");
        if (customer && (digits.length() < 10 || digits.length() > 13)) {
            throw new AppException("Enter a valid contact number so the owner can send a reminder.");
        }
        if (!customer && digits.length() > 0 && (digits.length() < 10 || digits.length() > 13)) {
            throw new AppException("Contact number should be 10–13 digits, or leave it blank.");
        }
    }
```

utangtracker.service/AuthService.java • isStrongPassword() • lines 255-268 • used by: OwnerFrame: Create Staff; RegisterFrame: Register +2

```java
    private boolean isStrongPassword(String password) {
        if (password == null || password.length() < 8 || password.length() > 12) {
            return false;
        }
        boolean hasUpper = false;
        boolean hasDigit = false;
        boolean hasSpecial = false;
        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
            else if (Character.isDigit(c)) hasDigit = true;
            else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
        }
        return hasUpper && hasDigit && hasSpecial;
    }
```

utangtracker.service/AuthService.java • requireOwner() • lines 270-274 • used by: OwnerFrame: Add to Utang; OwnerFrame: Create Staff +6

```
270     private void requireOwner() throws AppException {
271         if (Session.get() == null || Session.get().getRole() != Role.OWNER) {
272             throw new AppException("Only the owner can do that.");
273         }
274     }
```

utangtracker.service/AuthService.java • actor() • lines 276-278 • used by: OwnerFrame: Create Staff; OwnerFrame: Delete Customer +1

```
276     private String actor() {
277         return Session.get().getFirstName() + " " + Session.get().getLastName();
278     }
```


utangtracker.service/AuthService.java • isBlank() • lines 280-282 • used by: LoginFrame: Login; OwnerFrame: Create Staff +3
280 private boolean isBlank(String value) {
281 return value == null || value.trim().isEmpty();
282 }

utangtracker.service/UtangService.java • balanceOf() • lines 31-37 • used by: OwnerFrame: Pay Utang; OwnerFrame: Record Payment +1

```java
31    public double balanceOf(int customerId) throws AppException {
32        try {
33            return database.balanceOf(customerId);
34        } catch (SQLException ex) {
35            throw new AppException("Cannot compute the balance.");
36        }
37    }
```

utangtracker.service/UtangService.java • searchCustomers() • lines 39-46 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```
39      public List<CustomerBalance> searchCustomers(String search) throws AppException {
40          requireStaffOrOwner();
41          try {
42              return database.customersWithBalances(search);
43          } catch (SQLException ex) {
44              throw new AppException("Cannot search customers.");
45          }
46      }
```

utangtracker.service/UtangService.java • historyFor() • lines 48-55 • used by: OwnerFrame: table

```java
    public List<LedgerEntry> historyFor(int customerId) throws AppException {
        requireStaffOrOwner();
        try {
            return database.history(Integer.valueOf(customerId), null, false);
        } catch (SQLException ex) {
            throw new AppException("Cannot load that customer's history.");
        }
    }
```

utangtracker.service/UtangService.java • allHistory() • lines 57-64 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```
57    public List<LedgerEntry> allHistory(String search, boolean includeVoided) throws AppException {
58        requireOwner();
59        try {
60            return database.history(null, search, includeVoided);
61        } catch (SQLException ex) {
62            throw new AppException("Cannot load the ledger.");
63        }
64    }
```

utangtracker.service/UtangService.java • addUtang() • lines 66-79 • used by: OwnerFrame: Add to Utang; StaffFrame: Add to Utang

```
66      public LedgerEntry addUtang(int customerId, String item, double quantity, double unitPrice, String note) throws AppException {
67          requireStaffOrOwner();
68          if (item == null || item.trim().length() < 2) {
69              throw new AppException("Enter the item that was borrowed.");
70          }
71          if (quantity <= 0) {
72              throw new AppException("Quantity must be greater than zero.");
73          }
74          if (unitPrice <= 0) {
75              throw new AppException("Unit price must be greater than zero.");
76          }
77          double amount = Database.round(quantity * unitPrice);
78          return write(customerId, item.trim(), quantity, unitPrice, amount, "UTANG", note);
79      }
```

utangtracker.service/UtangService.java • acceptPayment() • lines 81-91 • used by: OwnerFrame: Pay Utang; OwnerFrame: Record Payment +1

```
81      public LedgerEntry acceptPayment(int customerId, double amount, String note) throws AppException {
82          requireStaffOrOwner();
83          if (amount <= 0) {
84              throw new AppException("Payment amount must be greater than zero.");
85          }
86          double balance = balanceOf(customerId);
87          if (amount - balance > 0.009) {
88              throw new AppException("Payment is larger than the unpaid balance (" + utangtracker.util.AppTime.peso(balance) + ").");
89          }
90          return write(customerId, "Cash payment", 1, amount, amount, "PAYMENT", note);
91      }
```

utangtracker.service/UtangService.java • kpis() • lines 104-111 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public KpiSnapshot kpis() throws AppException {
        requireOwner();
        try {
            return database.kpis();
        } catch (SQLException ex) {
            throw new AppException("Cannot load the dashboard.");
        }
    }
```

utangtracker.service/UtangService.java • auditLog() • lines 113-120 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public List<String[]> auditLog() throws AppException {
        requireOwner();
        try {
            return database.recentAudit();
        } catch (SQLException ex) {
            throw new AppException("Cannot load the audit log.");
        }
    }
```

utangtracker.service/UtangService.java • weekCollections() • lines 122-129 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```
122     public double weekCollections() throws AppException {
123         requireOwner();
124         try {
125             return database.collectionsSince(utangtracker.util.AppTime.daysAgo(7, "00:00:00"));
126         } catch (SQLException ex) {
127             throw new AppException("Cannot load weekly collections.");
128         }
129     }
```

utangtracker.service/UtangService.java • highestBalances() • lines 131-140 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public List<CustomerBalance> highestBalances() throws AppException {
        requireOwner();
        List<CustomerBalance> rows = searchCustomers("");
        java.util.Collections.sort(rows, new java.util.Comparator<CustomerBalance>() {
            public int compare(CustomerBalance a, CustomerBalance b) {
                return Double.compare(b.getBalance(), a.getBalance());
            }
        });
        return rows.size() > 5 ? rows.subList(0, 5) : rows;
    }
```

utangtracker.service/UtangService.java • staff() • lines 142-149 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```
142     public List<User> staff() throws AppException {
143         requireOwner();
144         try {
145             return database.listByRole(Role.STAFF);
146         } catch (SQLException ex) {
147             throw new AppException("Cannot load staff accounts.");
148         }
149     }
```

utangtracker.service/UtangService.java • write() • lines 157-176 • used by: OwnerFrame: Add to Utang; OwnerFrame: Pay Utang +3

```
157     private LedgerEntry write(int customerId, String item, double quantity, double unitPrice, double amount, String type, String note) throws AppException {
158         try {
159             User customer = database.findUser(customerId);
160             if (customer == null || customer.getRole() != Role.CUSTOMER || !customer.isActive()) {
161                 throw new AppException("Choose an active customer account.");
162             }
163             int id = database.addTransaction(customerId, Session.get().getId(), item, quantity, unitPrice, amount, type, note);
164             database.audit(Session.get().getId(), Session.get().getFirstName() + " " + Session.get().getLastName(),
165                     "CREATE", "CUST-" + customerId, type + " " + utangtracker.util.AppTime.peso(amount) + " " + item);
166             List<LedgerEntry> rows = database.history(Integer.valueOf(customerId), null, true);
167             for (LedgerEntry row : rows) {
168                 if (row.getId() == id) {
169                     return row;
170                 }
171             }
172             throw new AppException("Saved, but the new record could not be reloaded.");
173         } catch (SQLException ex) {
174             throw new AppException("Cannot save that record.");
175         }
176     }
```

utangtracker.service/UtangService.java • requireStaffOrOwner() • lines 184-189 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```
184     private void requireStaffOrOwner() throws AppException {
185         Role role = Session.get() == null ? null : Session.get().getRole();
186         if (role != Role.STAFF && role != Role.OWNER) {
187             throw new AppException("Only staff or the owner can record utang and payments.");
188         }
189     }
```

utangtracker.service/UtangService.java • requireOwner() • lines 191-195 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```
191     private void requireOwner() throws AppException {
192         if (Session.get() == null || Session.get().getRole() != Role.OWNER) {
193             throw new AppException("Only the owner can do that.");
194         }
195     }
```

utangtracker.db/Database.java • insertUser() • lines 205-227 • used by: OwnerFrame: Create Staff; RegisterFrame: Register +1

```java
    public int insertUser(String firstName, String lastName, String username, String password, String contact, Role role) throws SQLException {
        String salt = PasswordUtil.newSalt();
        String hash = PasswordUtil.hash(password, salt);
        String sql = "INSERT INTO users (first_name, last_name, username, password_hash, salt, contact, role, active, created_at) "
                + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
        try (PreparedStatement statement = connection.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            statement.setString(1, firstName.trim());
            statement.setString(2, lastName.trim());
            statement.setString(3, username.trim().toLowerCase());
            statement.setString(4, hash);
            statement.setString(5, salt);
            statement.setString(6, contact == null ? "" : contact.trim());
            statement.setString(7, role.name());
            int active = (role == Role.CUSTOMER) ? 0 : 1;
            statement.setInt(8, active);
            statement.setString(9, AppTime.now());
            statement.executeUpdate();
            try (ResultSet keys = statement.getGeneratedKeys()) {
                keys.next();
                return keys.getInt(1);
            }
        }
    }
```

utangtracker.db/Database.java • findByUsername() • lines 245-258 • used by: LoginFrame: Login

```java
public User findByUsername(String username) throws SQLException {
    String sql = "SELECT id, first_name, last_name, username, contact, role, active, password_hash, salt FROM users WHERE username = ?";
    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, username.trim().toLowerCase());
        try (ResultSet rs = statement.executeQuery()) {
            if (!rs.next()) {
                return null;
            }
            User user = mapUser(rs);
            user.setActive(rs.getInt("active") == 1);
            return user;
        }
    }
}
```

utangtracker.db/Database.java • credentials() • lines 260-271 • used by: LoginFrame: Login

```java
    public String[] credentials(String username) throws SQLException {
        String sql = "SELECT password_hash, salt, active, role FROM users WHERE username = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, username.trim().toLowerCase());
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return null;
                }
                return new String[] {rs.getString("password_hash"), rs.getString("salt"), String.valueOf(rs.getInt("active")), rs.getString("role")};
            }
        }
    }
```

utangtracker.db/Database.java • usernameExists() • lines 273-281 • used by: OwnerFrame: Create Staff; RegisterFrame: Register +1

```java
    public boolean usernameExists(String username) throws SQLException {
        String sql = "SELECT 1 FROM users WHERE username = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, username.trim().toLowerCase());
            try (ResultSet rs = statement.executeQuery()) {
                return rs.next();
            }
        }
    }
```

utangtracker.db/Database.java • findUser() • lines 283-294 • used by: OwnerFrame: Add to Utang; OwnerFrame: Create Staff +9

```java
    public User findUser(int id) throws SQLException {
        String sql = "SELECT id, first_name, last_name, username, contact, role, active FROM users WHERE id = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, id);
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return null;
                }
                return mapUser(rs);
            }
        }
    }
```

utangtracker.db/Database.java • listByRole() • lines 296-308 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```java
    public List<User> listByRole(Role role) throws SQLException {
        String sql = "SELECT id, first_name, last_name, username, contact, role, active FROM users WHERE role = ? ORDER BY last_name, first_name";
        List<User> users = new ArrayList<User>();
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, role.name());
            try (ResultSet rs = statement.executeQuery()) {
                while (rs.next()) {
                    users.add(mapUser(rs));
                }
            }
        }
        return users;
    }
```

utangtracker.db/Database.java • listPendingCustomers() • lines 310-320 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public List<User> listPendingCustomers() throws SQLException {
        String sql = "SELECT id, first_name, last_name, username, contact, role, active FROM users WHERE role = 'CUSTOMER' AND active = 0 ORDER BY created_at DESC";
        List<User> users = new ArrayList<User>();
        try (Statement statement = connection.createStatement();
             ResultSet rs = statement.executeQuery(sql)) {
            while (rs.next()) {
                users.add(mapUser(rs));
            }
        }
        return users;
    }
```

utangtracker.db/Database.java • updateCustomerInfo() • lines 322-331 • used by: OwnerFrame: Update Customer

```java
    public void updateCustomerInfo(int userId, String firstName, String lastName, String contact) throws SQLException {
        try (PreparedStatement statement = connection.prepareStatement(
                "UPDATE users SET first_name = ?, last_name = ?, contact = ? WHERE id = ? AND role = 'CUSTOMER'")) {
            statement.setString(1, firstName.trim());
            statement.setString(2, lastName.trim());
            statement.setString(3, contact == null ? "" : contact.trim());
            statement.setInt(4, userId);
            statement.executeUpdate();
        }
    }
```

utangtracker.db/Database.java • setActive() • lines 341-347 • used by: OwnerFrame: table

```java
    public void setActive(int userId, boolean active) throws SQLException {
        try (PreparedStatement statement = connection.prepareStatement("UPDATE users SET active = ? WHERE id = ?")) {
            statement.setInt(1, active ? 1 : 0);
            statement.setInt(2, userId);
            statement.executeUpdate();
        }
    }
```

utangtracker.db/Database.java • addTransaction() • lines 349-368 • used by: OwnerFrame: Add to Utang; OwnerFrame: Pay Utang +3

```java
    public int addTransaction(int customerId, int staffId, String item, double quantity, double unitPrice, double amount, String type, String note) throws SQLException {
        String sql = "INSERT INTO transactions (customer_id, staff_id, item_description, quantity, unit_price, amount, txn_type, note, created_at, voided) "
                + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, 0)";
        try (PreparedStatement statement = connection.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            statement.setInt(1, customerId);
            statement.setInt(2, staffId);
            statement.setString(3, item);
            statement.setDouble(4, quantity);
            statement.setDouble(5, unitPrice);
            statement.setDouble(6, round(amount));
            statement.setString(7, type);
            statement.setString(8, note == null ? "" : note.trim());
            statement.setString(9, AppTime.now());
            statement.executeUpdate();
            try (ResultSet keys = statement.getGeneratedKeys()) {
                keys.next();
                return keys.getInt(1);
            }
        }
    }
```

utangtracker.db/Database.java • deleteCustomer() • lines 377-397 • used by: OwnerFrame: Delete Customer

```java
    /** Permanently removes a customer account and all of their transactions. */
    public void deleteCustomer(int userId) throws SQLException {
        boolean previous = connection.getAutoCommit();
        connection.setAutoCommit(false);
        try {
            try (PreparedStatement t = connection.prepareStatement("DELETE FROM transactions WHERE customer_id = ?")) {
                t.setInt(1, userId);
                t.executeUpdate();
            }
            try (PreparedStatement u = connection.prepareStatement("DELETE FROM users WHERE id = ? AND role = 'CUSTOMER'")) {
                u.setInt(1, userId);
                u.executeUpdate();
            }
            connection.commit();
        } catch (SQLException ex) {
            connection.rollback();
            throw ex;
        } finally {
            connection.setAutoCommit(previous);
        }
    }
```

utangtracker.db/Database.java • balanceOf() • lines 399-411 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```java
    public double balanceOf(int customerId) throws SQLException {
        String sql = "SELECT "
                + "COALESCE(SUM(CASE WHEN txn_type = 'UTANG' AND voided = 0 THEN amount ELSE 0 END), 0) - "
                + "COALESCE(SUM(CASE WHEN txn_type = 'PAYMENT' AND voided = 0 THEN amount ELSE 0 END), 0) "
                + "AS balance FROM transactions WHERE customer_id = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, customerId);
            try (ResultSet rs = statement.executeQuery()) {
                rs.next();
                return round(rs.getDouble("balance"));
            }
        }
    }
```

utangtracker.db/Database.java • history() • lines 413-455 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +6

```java
    public List<LedgerEntry> history(Integer customerId, String search, boolean includeVoided) throws SQLException {
        StringBuilder sql = new StringBuilder();
        sql.append("SELECT t.id, t.customer_id, c.first_name AS customer_first, c.last_name AS customer_last, t.staff_id, ");
        sql.append("s.first_name AS staff_first, s.last_name AS staff_last, t.item_description, t.quantity, t.unit_price, t.amount, ");
        sql.append("t.txn_type, t.note, t.created_at, t.voided ");
        sql.append("FROM transactions t ");
        sql.append("JOIN users c ON c.id = t.customer_id ");
        sql.append("LEFT JOIN users s ON s.id = t.staff_id ");
        sql.append("WHERE 1 = 1 ");
        if (!includeVoided) {
            sql.append("AND t.voided = 0 ");
        }
        if (customerId != null) {
            sql.append("AND t.customer_id = ? ");
        }
        if (search != null && search.trim().length() > 0) {
            sql.append("AND (c.first_name LIKE ? OR c.last_name LIKE ? OR t.item_description LIKE ? OR IFNULL(t.note, '') LIKE ? OR IFNULL(s.first_name, '') LIKE ? OR IFNULL(s.last_name, '') LIKE ?) ");
        }
        sql.append("ORDER BY t.created_at DESC, t.id DESC");

        List<LedgerEntry> rows = new ArrayList<LedgerEntry>();
        try (PreparedStatement statement = connection.prepareStatement(sql.toString())) {
            int index = 1;
            if (customerId != null) {
                statement.setInt(index++, customerId);
            }
            if (search != null && search.trim().length() > 0) {
                String like = "%" + search.trim() + "%";
                statement.setString(index++, like);
                statement.setString(index++, like);
                statement.setString(index++, like);
                statement.setString(index++, like);
                statement.setString(index++, like);
                statement.setString(index, like);
            }
            try (ResultSet rs = statement.executeQuery()) {
                while (rs.next()) {
                    rows.add(mapEntry(rs));
                }
            }
        }
        return rows;
    }
```

utangtracker.db/Database.java • customersWithBalances() • lines 457-475 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```java
    public List<CustomerBalance> customersWithBalances(String search) throws SQLException {
        List<User> customers = listByRole(Role.CUSTOMER);
        List<CustomerBalance> rows = new ArrayList<CustomerBalance>();
        String needle = search == null ? "" : search.trim().toLowerCase();
        for (User customer : customers) {
            if (!customer.isActive()) {
                continue;
            }
            String haystack = ("cust-" + customer.getId() + " " + customer.getFirstName() + " " + customer.getLastName()
                    + " " + customer.getUsername() + " " + customer.getContact()).toLowerCase();
            if (needle.length() > 0 && haystack.indexOf(needle) < 0) {
                continue;
            }
            double balance = balanceOf(customer.getId());
            String last = lastActivity(customer.getId());
            rows.add(new CustomerBalance(customer, balance, last));
        }
        return rows;
    }
```

utangtracker.db/Database.java • audit() • lines 477-489 • used by: OwnerFrame: Add to Utang; OwnerFrame: Create Staff +10

```java
    public void audit(int actorId, String actorName, String action, String target, String detail) {
        String sql = "INSERT INTO audit_log (actor_id, actor_name, action, target, detail, created_at) VALUES (?, ?, ?, ?, ?, ?)";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, actorId);
            statement.setString(2, actorName);
            statement.setString(3, action);
            statement.setString(4, target);
            statement.setString(5, detail);
            statement.setString(6, AppTime.now());
            statement.executeUpdate();
        } catch (SQLException ignored) {
        }
    }
```

utangtracker.db/Database.java • recentAudit() • lines 491-500 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public List<String[]> recentAudit() throws SQLException {
        List<String[]> rows = new ArrayList<String[]>();
        try (Statement statement = connection.createStatement();
             ResultSet rs = statement.executeQuery("SELECT created_at, actor_name, action, target, detail FROM audit_log ORDER BY id DESC LIMIT 200")) {
            while (rs.next()) {
                rows.add(new String[] {rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4), rs.getString(5)});
            }
        }
        return rows;
    }
```

utangtracker.db/Database.java • collectionsSince() • lines 502-511 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public double collectionsSince(String since) throws SQLException {
        String sql = "SELECT COALESCE(SUM(amount), 0) FROM transactions WHERE txn_type = 'PAYMENT' AND voided = 0 AND created_at >= ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, since);
            try (ResultSet rs = statement.executeQuery()) {
                rs.next();
                return round(rs.getDouble(1));
            }
        }
    }
```

utangtracker.db/Database.java • resetPassword() • lines 519-565 • used by: ResetPinFrame: reset

```java
    /**
     * PIN-based password reset. Customer accounts only. After 5 wrong tries the
     * account's PIN reset is locked for 15 minutes.
     */
    public int resetPassword(String username, String pin, String newPassword) throws SQLException {
        int id;
        String role;
        String storedPin;
        int attempts;
        String lockedUntil;
        String sql = "SELECT id, role, recovery_pin, pin_attempts, pin_locked_until FROM users WHERE username = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, username.trim().toLowerCase());
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return RESET_BAD;
                }
                id = rs.getInt("id");
                role = rs.getString("role");
                storedPin = rs.getString("recovery_pin");
                attempts = rs.getInt("pin_attempts");
                lockedUntil = rs.getString("pin_locked_until");
            }
        }
        if (lockedUntil != null && lockedUntil.compareTo(AppTime.now()) > 0) {
            return RESET_LOCKED;
        }
        boolean allowed = Role.CUSTOMER.name().equals(role) && PasswordUtil.matchesPin(pin, storedPin);
        if (!allowed) {
            attempts++;
            if (attempts >= MAX_PIN_TRIES) {
                setPinState(id, 0, AppTime.minutesFromNow(LOCK_MINUTES));
                return RESET_LOCKED;
            }
            setPinState(id, attempts, null);
            return RESET_BAD;
        }
        String salt = PasswordUtil.newSalt();
        try (PreparedStatement update = connection.prepareStatement(
                "UPDATE users SET password_hash = ?, salt = ?, pin_attempts = 0, pin_locked_until = NULL WHERE id = ?")) {
            update.setString(1, PasswordUtil.hash(newPassword, salt));
            update.setString(2, salt);
            update.setInt(3, id);
            update.executeUpdate();
        }
        return RESET_OK;
    }
```

utangtracker.db/Database.java • setPinState() • lines 567-575 • used by: OwnerFrame: Reset User PIN; ResetPinFrame: reset

```
567     private void setPinState(int userId, int attempts, String lockedUntil) throws SQLException {
568         try (PreparedStatement statement = connection.prepareStatement(
569                 "UPDATE users SET pin_attempts = ?, pin_locked_until = ? WHERE id = ?")) {
570             statement.setInt(1, attempts);
571             statement.setString(2, lockedUntil);
572             statement.setInt(3, userId);
573             statement.executeUpdate();
574         }
575     }
```

utangtracker.db/Database.java • saveRecoveryPin() • lines 577-583 • used by: OwnerFrame: Reset User PIN; RegisterFrame: Register +1

```java
    public void saveRecoveryPin(int userId, String pin) throws SQLException {
        try (PreparedStatement statement = connection.prepareStatement("UPDATE users SET recovery_pin = ? WHERE id = ?")) {
            statement.setString(1, PasswordUtil.isHashedPin(pin) ? pin : PasswordUtil.hashPin(pin));
            statement.setInt(2, userId);
            statement.executeUpdate();
        }
    }
```

utangtracker.db/Database.java • findCustomerByContact() • lines 585-601 • used by: LoginFrame: Login

```java
    /** Finds an active customer by contact number (digits only). */
    public User findCustomerByContact(String digits) throws SQLException {
        if (digits == null || digits.isEmpty()) {
            return null;
        }
        String sql = "SELECT id, first_name, last_name, username, contact, role, active FROM users "
                + "WHERE REPLACE(REPLACE(REPLACE(contact, ' ', ''), '-', ''), '+', '') = ? AND role = 'CUSTOMER' LIMIT 1";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, digits);
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return null;
                }
                return mapUser(rs);
            }
        }
    }
```

utangtracker.db/Database.java • matchesRecoveryPin() • lines 603-616 • used by: LoginFrame: Login

```java
    /** Returns true if the given plain PIN matches the stored recovery PIN for the user. */
    public boolean matchesRecoveryPin(int userId, String pin) throws SQLException {
        String sql = "SELECT recovery_pin FROM users WHERE id = ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, userId);
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return false;
                }
                String stored = rs.getString("recovery_pin");
                return PasswordUtil.matchesPin(pin, stored);
            }
        }
    }
```

utangtracker.db/Database.java • resetPinByContact() • lines 618-639 • used by: OwnerFrame: Reset User PIN

```java
    /** Finds an active customer by contact number and resets their recovery PIN to the given value. */
    public User resetPinByContact(String contact, String newPin) throws SQLException {
        String digits = contact == null ? "" : contact.replaceAll("[^0-9]", "");
        if (digits.isEmpty()) {
            return null;
        }
        String sql = "SELECT id, first_name, last_name, username, contact, role, active FROM users "
                + "WHERE REPLACE(REPLACE(REPLACE(contact, ' ', ''), '-', ''), '+', '') = ? AND role = 'CUSTOMER' AND active = 1 LIMIT 1";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, digits);
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return null;
                }
                User user = mapUser(rs);
                saveRecoveryPin(user.getId(), newPin);
                // Also clear any PIN lock so they can use it immediately
                setPinState(user.getId(), 0, null);
                return user;
            }
        }
    }
```

utangtracker.db/Database.java • lastActivity() • lines 641-652 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```java
    public String lastActivity(int customerId) throws SQLException {
        String sql = "SELECT created_at FROM transactions WHERE customer_id = ? AND voided = 0 ORDER BY created_at DESC, id DESC LIMIT 1";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, customerId);
            try (ResultSet rs = statement.executeQuery()) {
                if (!rs.next()) {
                    return "—";
                }
                return rs.getString(1);
            }
        }
    }
```

utangtracker.db/Database.java • kpis() • lines 654-681 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +4

```java
    public KpiSnapshot kpis() throws SQLException {
        double unpaid = 0;
        int withUtang = 0;
        List<User> customers = listByRole(Role.CUSTOMER);
        for (User customer : customers) {
            double balance = balanceOf(customer.getId());
            if (balance > 0.009) {
                unpaid += balance;
                withUtang++;
            }
        }
        int todayCount = 0;
        double collected = 0;
        String prefix = AppTime.todayPrefix();
        String sql = "SELECT txn_type, amount FROM transactions WHERE voided = 0 AND created_at LIKE ?";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, prefix + "%");
            try (ResultSet rs = statement.executeQuery()) {
                while (rs.next()) {
                    todayCount++;
                    if ("PAYMENT".equals(rs.getString("txn_type"))) {
                        collected += rs.getDouble("amount");
                    }
                }
            }
        }
        return new KpiSnapshot(round(unpaid), withUtang, todayCount, round(collected));
    }
```


utangtracker.db/Database.java • mapUser() • lines 683-692 • used by: LoginFrame: Login; OwnerFrame: Add to Utang +11
683 private User mapUser(ResultSet rs) throws SQLException {
684 return new User(
685 rs.getInt("id"),
686 rs.getString("first_name"),
687 rs.getString("last_name"),
688 rs.getString("username"),
689 rs.getString("contact"),
690 Role.from(rs.getString("role")),
691 rs.getInt("active") == 1);
692 }

utangtracker.db/Database.java • mapEntry() • lines 694-715 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +6

```java
    private LedgerEntry mapEntry(ResultSet rs) throws SQLException {
        LedgerEntry entry = new LedgerEntry();
        entry.setId(rs.getInt("id"));
        entry.setCustomerId(rs.getInt("customer_id"));
        entry.setCustomerFirstName(rs.getString("customer_first"));
        entry.setCustomerLastName(rs.getString("customer_last"));
        int staffId = rs.getInt("staff_id");
        if (!rs.wasNull()) {
            entry.setStaffId(Integer.valueOf(staffId));
        }
        entry.setStaffFirstName(rs.getString("staff_first"));
        entry.setStaffLastName(rs.getString("staff_last"));
        entry.setItemDescription(rs.getString("item_description"));
        entry.setQuantity(rs.getDouble("quantity"));
        entry.setUnitPrice(rs.getDouble("unit_price"));
        entry.setAmount(rs.getDouble("amount"));
        entry.setType(rs.getString("txn_type"));
        entry.setNote(rs.getString("note"));
        entry.setCreatedAt(rs.getString("created_at"));
        entry.setVoided(rs.getInt("voided") == 1);
        return entry;
    }
```

utangtracker.db/Database.java • round() • lines 717-719 • used by: OwnerFrame: Add to Utang; OwnerFrame: Delete Customer +7

```
717     public static double round(double amount) {
718         return Math.round(amount * 100.0) / 100.0;
719     }
```

utangtracker.model/Role.java • from() • lines 21-23 • used by: LoginFrame: Login; OwnerFrame: Add to Utang +11

```
21      public static Role from(String value) {
22          return Role.valueOf(value.trim().toUpperCase());
23      }
```

utangtracker.util/PasswordUtil.java • newSalt() • lines 13-17 • used by: OwnerFrame: Create Staff; OwnerFrame: Reset User PIN +3

```java
    public static String newSalt() {
        byte[] bytes = new byte[16];
        RANDOM.nextBytes(bytes);
        return toHex(bytes);
    }
```

utangtracker.util/PasswordUtil.java • hash() • lines 19-27 • used by: LoginFrame: Login; OwnerFrame: Create Staff +4

```java
    public static String hash(String password, String salt) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashed = digest.digest((salt + ":" + password).getBytes(StandardCharsets.UTF_8));
            return toHex(hashed);
        } catch (Exception ex) {
            throw new IllegalStateException("Cannot hash password", ex);
        }
    }
```

utangtracker.util/PasswordUtil.java • matches() • lines 29-31 • used by: LoginFrame: Login; ResetPinFrame: reset

```
29      public static boolean matches(String password, String salt, String expectedHash) {
30          return hash(password, salt).equals(expectedHash);
31      }
```

utangtracker.util/PasswordUtil.java • hashPin() • lines 33-37 • used by: OwnerFrame: Reset User PIN; RegisterFrame: Register +1

```java
    /** Hashes a recovery PIN. Stored as "salt$hash" so it is never kept in plain text. */
    public static String hashPin(String pin) {
        String salt = newSalt();
        return salt + "$" + hash(pin, salt);
    }
```

utangtracker.util/PasswordUtil.java • isHashedPin() • lines 39-41 • used by: LoginFrame: Login; OwnerFrame: Reset User PIN +3

```
39      public static boolean isHashedPin(String stored) {
40          return stored != null && stored.indexOf('$') > 0;
41      }
```

utangtracker.util/PasswordUtil.java • matchesPin() • lines 43-49 • used by: LoginFrame: Login; ResetPinFrame: reset

```java
    public static boolean matchesPin(String pin, String stored) {
        if (pin == null || !isHashedPin(stored)) {
            return false;
        }
        int split = stored.indexOf('$');
        return matches(pin, stored.substring(0, split), stored.substring(split + 1));
    }
```

utangtracker.util/PasswordUtil.java • toHex() • lines 51-57 • used by: LoginFrame: Login; OwnerFrame: Create Staff +4

```java
    private static String toHex(byte[] bytes) {
        StringBuilder builder = new StringBuilder(bytes.length * 2);
        for (byte value : bytes) {
            builder.append(String.format("%02x", value));
        }
        return builder.toString();
    }
```